

Ana Julia Castelo Branco
Leonardo Lotério de Lima
Lucas Honorato
Marina Rosa Oliveira
Miguel Angelo Dufloth Filho
Miguel Angel Balldares Huertas

O software vai sobreviver?

Análise inicial do caso e riscos futuros

Ana Julia Castelo Branco
Leonardo Lotério de Lima
Lucas Honorato
Marina Rosa Oliveira
Miguel Angelo Dufloth Filho
Miguel Angel Baldares Huertas

O software vai sobreviver?

Análise inicial do caso e riscos futuros

Atividade apresentada como análise inicial de um sistema de gestão de armários para academias, com ênfase em arquitetura, riscos técnicos, dívida técnica e evolução de software.

Sumário

1	VISÃO GERAL DO SISTEMA	3
1.1	Objetivo	3
1.2	Público-alvo	3
1.3	Tecnologias identificadas	3
2	ANÁLISE ARQUITETURAL	4
2.1	Estrutura do projeto	4
2.2	Dependências	4
2.3	Componentes principais	4
3	RISCOS TÉCNICOS	5
4	DÍVIDAS TÉCNICAS IDENTIFICADAS	6
5	PROGNÓSTICO DE EVOLUÇÃO	7
5.1	Sobrevivência do sistema em cinco anos	7
5.2	Principais ameaças	7
	REFERÊNCIAS	8

1 Visão geral do sistema

1.1 Objetivo

O sistema tem como objetivo gerenciar armários rotativos de uso diário em academias. A aplicação deverá permitir o registro de entrada (*check-in*) e de saída (*check-out*) dos usuários, assegurando que cada aluno ocupe somente um armário por vez.

1.2 Público-alvo

O público-alvo é composto por alunos da academia, na condição de usuários finais, e por recepcionistas e administradores responsáveis pelo gerenciamento dos armários.

1.3 Tecnologias identificadas

As tecnologias definidas inicialmente para o desenvolvimento do sistema são:

- *front-end*: React com Vite;
- *back-end*: FastAPI;
- banco de dados: SQLite;
- autenticação: JSON Web Token (JWT);
- infraestrutura: Docker Compose;
- controle de versão e hospedagem do código-fonte: GitHub.

2 Análise arquitetural

2.1 Estrutura do projeto

O projeto adotará uma arquitetura cliente-servidor, com separação entre uma interface de programação de aplicações (API) RESTful no *back-end* e uma aplicação de página única (*Single Page Application* — SPA) no *front-end*. Essa divisão busca reduzir o acoplamento e facilitar a manutenção e a evolução dos componentes. O Docker Compose será utilizado para padronizar a execução dos serviços nos ambientes de desenvolvimento.

2.2 Dependências

As principais dependências identificadas são:

- banco de dados relacional, para garantir as propriedades de atomicidade, consistência, isolamento e durabilidade (ACID) nas transações de entrada e saída;
- ferramenta para documentação e teste da API, como Swagger ou Postman;
- bibliotecas ou *frameworks* para gerenciamento de rotas, autenticação por JWT e acesso ao banco de dados, por meio de mapeamento objeto-relacional (ORM) ou de um construtor de consultas (*query builder*);
- Docker e Docker Compose, para configuração e execução padronizada dos serviços da aplicação.

2.3 Componentes principais

Os componentes centrais previstos para o sistema são:

- módulo de autenticação e autorização de usuários;
- módulo de gestão de armários, responsável pelo cadastro, pela listagem e pela atualização do estado dos armários;
- módulo de ocupação, responsável pelas operações de entrada e saída e pela validação das regras de negócio.

3 Riscos técnicos

A Tabela 1 sintetiza os principais riscos técnicos identificados, bem como seu impacto, sua probabilidade e sua prioridade de tratamento.

Tabela 1 – Riscos técnicos identificados

Risco	Impacto	Probabilidade	Prioridade
Falha no controle de concorrência do banco de dados, permitindo que dois alunos reservem simultaneamente o mesmo armário.	Alto	Média	Alta
Ausência de testes automatizados para a regra que limita a ocupação a um armário por aluno, ocasionando falhas a serem herdadas pela equipe da N2.	Alto	Alta	Alta
Forte acoplamento entre as regras de negócio e os controladores da API, dificultando a refatoração e a auditoria do código por outras equipes.	Médio	Alta	Média
Armazenamento ou validação inadequados dos tokens JWT, possibilitando acesso não autorizado ao sistema.	Alto	Média	Alta

Fonte: Elaborado pelos autores (2026).

4 Dívidas técnicas identificadas

A Tabela 2 apresenta as dívidas técnicas inicialmente reconhecidas no projeto.

Tabela 2 – Dívidas técnicas identificadas

Item	Categoria	Gravidade
Ausência de um fluxo automatizado de integração e entrega contínuas (CI/CD) para a execução de testes e a implantação do sistema, previsto somente para a N3.	Infraestrutura	Baixa
Tratamento genérico de erros na API, dificultando o diagnóstico de falhas e a integração da próxima equipe ao projeto.	Código e projeto	Média
Ausência de rastreabilidade e de histórico de utilização dos armários, uma vez que a versão atual se concentra apenas no estado presente.	Arquitetura	Média

Fonte: Elaborado pelos autores (2026).

5 Prognóstico de evolução

5.1 Sobrevivência do sistema em cinco anos

Em sua versão N1, o sistema provavelmente não sobreviveria por cinco anos sem manutenção evolutiva. Segundo Lehman (1980), um programa inserido em um ambiente real precisa ser continuamente adaptado; caso contrário, sua utilidade tende a diminuir. Para permanecer relevante, o sistema deverá incorporar requisitos futuros, como liberação remota de armários, controle de uso fora do horário de funcionamento e cobrança de taxas por pernoite. Essas alterações exigirão refatoração arquitetural, redução da dívida técnica acumulada e adoção de práticas automatizadas de integração e entrega contínuas.

5.2 Principais ameaças

As principais ameaças à continuidade do sistema são:

1. **Deterioração do código:** a transferência do software entre equipes sem documentação adequada aumenta a probabilidade de introdução de indícios de baixa qualidade no código (*code smells*).
2. **Evolução das regras de negócio:** a futura cobrança de taxas por pernoite acrescentará complexidade financeira, fiscal e transacional não contemplada pela arquitetura inicial da N1.
3. **Acúmulo de dívida técnica:** a metáfora proposta por Cunningham (1992) indica que decisões de implementação tomadas para acelerar uma entrega podem gerar custos adicionais em alterações futuras. Caso a equipe da N2 não revise e refatore o sistema, sua evolução até a reengenharia prevista para a N3 poderá ser comprometida.

Referências

CUNNINGHAM, W. The WyCash portfolio management system. In: *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications*. New York: Association for Computing Machinery, 1992. p. 29–30.

LEHMAN, M. M. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, v. 68, n. 9, p. 1060–1076, set. 1980.